

中国研究生操作系统开源创新大赛

项目说明书

作品名称:	<u>KylinBootLab</u>
参赛队伍:	<u>直觉理性</u>
完成日期:	<u>2026年7月</u>

目录

第 1 章 绪 论.....	3
1.1 目的与意义	3
1.2 项目背景	3
第 2 章 技术理论.....	5
2.1 设计思想	5
2.2 技术路线	5
2.3 代码原创说明	7
第 3 章 项目介绍.....	8
3.1 功能介绍	8
3.1.1 SnapCollect 采集模块	8
3.1.2 Observer 观测器模块	8
3.1.3 Analyze 分析模块	9
3.1.4 Optimize 优化验证模块.....	10
3.1.5 BootAgent AI 辅助模块.....	10
3.1.6 Cross-Distro 跨发行版适配模块	11
3.2 软件界面	11
第 4 章 实验测试.....	16
4.1 测试说明	16
4.2 测试结果	17
第 5 章 实现难点说明.....	22
第 6 章 总 结.....	25
参 考.....	26

第 1 章 绪 论

1.1 目的与意义

KylinBootLab 面向 Linux 桌面操作系统的启动性能分析与优化领域，提供从数据采集、因果建模到实验验证的全链路自动化工具链。

作品设计目的：通过客观数据驱动的方法，替代传统的"经验式"启动优化模式，将优化过程从手工推测转变为严格的科学实验验证[1]。传统启动优化依赖开发者对系统服务的经验判断，缺乏系统性的瓶颈定位和统计置信的优化验证。KylinBootLab 通过因果图分析量化每个服务对整体启动延迟的贡献，通过 ABBA 实验协议以统计置信的方式判定优化有效性，填补了该领域的工具空白[2]。

应用意义：一方面为 openKylin 等国产操作系统提供系统化的启动性能诊断手段，帮助识别和消除启动路径上的关键瓶颈；另一方面通过跨发行版适配器架构，验证了分析管道在 openKylin、Ubuntu 22.04 LTS、Fedora 41 三个不同发行版上的通用性，为 Linux 生态的启动优化工作提供了可复用的方法论参考和开源工具基础。

1.2 项目背景

现有启动分析工具以 systemd-analyze 为代表[3,4,5]，能够提供启动时间分解、服务耗时排序和关键链 critical-chain，但存在三个核心缺陷：

(1) 仅提供静态数据：systemd-analyze 输出单次启动的测量结果，不具备优化方案的实验验证能力。用户需要手动执行优化操作、手动重启、手动对比结果，缺乏自动化的 A/B 对比框架。

(2) 缺乏统计推断：单次启动测量受系统噪声例如磁盘缓存状态、ASLR、systemd 服务启动的竞态条件影响，重复测量之间存在不可忽略的方差。没有任何工具提供跨多次重复启动的统计显著性检验。

(3) 发行版耦合：各发行版的 initramfs 构建工具链（initramfs-tools vs dracut）、显示管理器（LightDM vs GDM）、系统服务命名规范存在差异，导致分析脚本无法跨发行版复用。

KylinBootLab 的创新之处体现在以下四个方面：

(1) 首次将 ABBA 实验协议引入 Linux 启动优化[6]，通过 18 次冷启动（2 warmup + 4×4 块排列）+ Bootstrap CI 10000 次重采样，实现统计置信的优化判定，将启动优化从"经验推测"提升为"科学实验"。

(2) 基于 Linux uinput 子系统实现无人值守自动登录观测器，通过纯状态机架构消

除人机交互的时序不确定性，在每次冷启动中精确记录 greeter 就绪、登录注入、PAM 会话开启和桌面可用性四个关键时间点。

（3）DOT 依赖图解析 + 拓扑排序动态规划 $O(V+E)$ 关键路径计算 + 三维瓶颈评分（ $\text{blame} \times \text{slack_penalty} \times \text{criticality}$ ），实现对数百个 systemd 单元构成的复杂依赖网络的系统级瓶颈定位和优化优先级排序。

（4）跨发行版适配器架构封装 initramfs 工具链差异、显示管理器差异和服务名称映射，在 openKylin 2.0 SP2、Ubuntu 22.04 LTS、Fedora 41 三个发行版上零代码修改完成基线采集与 ABBA 实验验证。

第2章 技术理论

2.1 设计思想

本作品遵循"数据驱动决策"与"科学实验方法论"两大核心设计思想。

数据驱动决策：优化候选方案不依赖开发者直觉，而是基于因果图分析产生的瓶颈评分自动排序[7]。瓶颈评分公式为 $\text{blame} \times \text{slack_penalty} \times \text{criticality}$ ，其中 **blame** 为服务自身耗时，**slack_penalty** 惩罚不在关键路径上的节点，**slack** 越大，惩罚系数越接近 0，节点得分越低。**criticality** 对关键路径上节点权重为 1.0，非关键路径上为 0.0，在公式中完全归零。系统自动从数百个候选服务中筛选出高价值优化目标，优先实验评分最高的组合方案。

科学实验方法论：每项优化候选方案需通过 ABBA 实验协议的严格验证。ABBA 是一种平衡排列的 A/B 实验设计，包括 18 次冷启动，一共分为 2 次预热（消除首次启动的系统状态建立开销）+ 4 个 ABBA 块。每个块内按 A-B-B-A 顺序交替执行对照组

（baseline snapshot）和实验组（优化 applied）的冷启动，抵消时间趋势（系统升温、磁盘碎片化积累等）。Bootstrap 非参数重采样 10000 次计算中位改善的 95% 置信区间，避免对数据分布的任何参数假设。校准基准（bare vs benchmark 模式对比）确认观测器额外引入的测量误差在可忽略范围内。

架构设计思想：管道式处理架构将系统分为五个独立阶段。每个阶段通过 JSON Schema 约束的数据契约与上下游通信，可独立测试、独立替换。适配器模式在采集管道中抽象发行版差异，实现对不同 Linux 发行版的透明支持[8]。不可变 RunStore 确保所有实验数据仅追加写入、不可修改，提供完整的审计追溯能力。

2.2 技术路线

全链路技术路线由五层构成，具体结构如图 1 所示，系统由五层管道构成。数据从底层 Rust 探针采集开始，经四阶段 TOCTOU 安全管道入库。具体地，每一层的具体职责和功能详细介绍如下：



图 1: KylinBootLab 分析管道架构

(1) 数据采集层 (Probe)：采用 Rust 1.85 编写探针 kbl-bootprob)，运行于目标 Linux 系统的 systemd 服务上下文中。捕获 systemd-analyze time，包含内核/initrd/用户空间三段分解、所有服务的启动耗时排序 systemd-analyze blame 和 Graphviz DOT 格式的完整依赖图 systemd-analyze dot。通过 JSONL 事件流记录观测器的就绪状态变化序列，输出结构化 JSON 清单 probe-manifest.json，包含 schema_version、exit_code、boot_id 和所有产物文件的 SHA-256 校验和。

(2) 观测器层 (Observer)：Rust 实现纯状态机驱动的自动化观测系统。状态机接受 Signal 枚举输入，包括 Journal、UnitActive、UinputReady、LoginInjected、UsableResult、Tick 等，输出 ReadinessEvent 序列包括 observer_started、greeter_ready、login_injected、session_opened、sentinel_launched、usable。核心机制包括：uinput 键盘模拟自动登录注入 (/dev/uinput 设备 + EV_KEY 事件)、journalctl -o json 日志流实时解析、AT-SPI D-Bus 桌面可用性探测。整个状态机无 I/O 等待（所有阻塞在外部 loop 中）、无实时时钟依赖（使用 monotonic Instant），行为完全可单元测试。

(3) 分析管道层 (Analyze)：Python 3.12 + 纯 Python 构建有向因果图。自研 DOT 解析器解析 systemd-analyze dot 输出，过滤 systemd 内部节点，仅保留 service/target/device 三类节点。拓扑排序后从源点向汇点动态规划计算每个节点的最早完成时间，反向 DP 得到最晚完成时间和松弛时间。瓶颈评分 = $\text{blame} \times (1 + 1/\text{slack}) \times \text{criticality}$ 。支持跨运行的图结构对比 (GraphDiff)，识别新增服务和依赖变化。

(4) 实验验证层 (Optimize)：使用 Python 实现 ABBA 协议。ProfileExecutor 通过 SSH 远端执行优化操作如 `systemctl mask/unmask`、内核参数修改等，VixPower 封装 VMware `vmrun` 命令管理 VM 快照创建/恢复和冷启动循环。ABBAPower 包装器保护 ABBA Profile 的快照不被自动恢复覆盖。Bootstrap CI 使用 `numpy` 实现 10000 次重采样，三级判定门控：PROMISING、ACCEPTED、REJECTED。

(5) Agent 辅助层 (BootAgent)：在本地部署 Qwen2.5-Coder-7B，并利用 CPU 进行推理，四角色 TOML 技能定义流水线——Trace Analyst 负责读取瓶颈报告和因果图，识别异常模式和性能瓶颈、Source Investigator 负责检查 `systemd unit` 文件配置问题，提出具体修改建议、Experiment Designer 负责生成结构化优化实验方案，定义 falsification 条件和回滚步骤、Safety Critic 负责评估风险评分和功能回归可能性，输出 APPROVE/REVIEW/REJECT 建议。支持 `kbl agent` 命令行交互式操作。

2.3 代码原创说明

本作品全部 Python 和 Rust 源代码均为原创实现，未引用任何外部源代码。第三方开源依赖库的使用遵循各项目的开源许可协议：

Python 依赖：typer (CLI 框架, MIT)、pydantic (数据验证, MIT)、jinja2 (模板引擎, BSD-3-Clause)、numpy (数值计算, BSD-3-Clause)、jsonschema (JSON Schema 验证, MIT)、requests (HTTP 客户端, Apache-2.0)。

Rust 依赖：serde/serde_json (序列化框架, MIT/Apache-2.0)、clap (CLI 参数解析, MIT/Apache-2.0)、nix (Linux 系统调用安全封装, MIT)、anyhow (错误处理, MIT/Apache-2.0)、uuid (唯一标识符生成, MIT/Apache-2.0)、input-linux (uinput 操作, MIT)、sha2 (SHA-256, MIT/Apache-2.0)、chrono (时间处理, MIT/Apache-2.0)、toml (TOML 解析, MIT/Apache-2.0)。

仪表板依赖：React 19 (UI 框架, MIT)、Vite 8 (构建工具, MIT)、Recharts 2 (图表库, MIT)、Tailwind CSS 4 (样式框架, MIT)。

所有第三方库均来自官方包仓库 PyPI、crates.io、npm，代码未经过修改。作品的独创性体现在系统架构设计、ABBA 实验协议实现、因果图分析算法和跨发行版适配器上。

第 3 章 项目介绍

3.1 功能介绍

KylinBootLab 是一个全链路 Linux 启动性能分析与优化平台，以 CLI 工具为核心入口，提供 11 个子命令，涵盖数据采集、分析、实验、AI 辅助四大领域。系统由以下核心模块构成，按数据流向下游依次排列：

数据流向：探针采集（snapshot → ingest）→ 基线报告（report）→ 因果图分析（analyze）→ 优化规划（optimize plan）→ ABBA 实验（optimize run）→ 证据展示（dashboard）。

3.1.1 SnapCollect 采集模块

SnapCollect 是启动数据采集模块，由 Rust 探针 kbl-bootprobe 实现。探针运行在目标 Linux 系统上，以 systemd 服务形式自动在系统启动早期执行。**核心功能包括：**

①systemd-analyze time 调用：采集内核 kernel 加载时间、initramfs 初始化时间和用户空间 userspace 启动时间三段分解，提供启动时间的宏观视图。

②systemd-analyze blame 调用：采集所有 systemd 服务的启动耗时排序列表，精确到毫秒级别，为瓶颈识别提供初步数据。

③systemd-analyze dot 调用：生成 Graphviz DOT 格式的完整 systemd 依赖图，包含 service、target、device、mount、socket、slice 等所有单元及其依赖关系边，是因果图分析的原始数据来源。

输出结构化 JSON 清单 probe-manifest.json，包含 协议版本 schema_version、采集退出码 exit_code、内核分配的启动唯一标识 boot_id 和所有产物文件的 SHA-256 校验和。支持单次完整采集 snapshot 和多次重复采集 calibration 两种模式。探测脚本通过 SCP 从控制主机远程执行，结果自动回传至 RunStore 完成四阶段数据入库，即 validate → index → store → finalize。

3.1.2 Observer 观测器模块

Observer 是实现无人值守冷启动计时自动化观测的核心模块。它作为 Type=simple 的 systemd 服务在目标系统启动时自动运行，按以下状态机执行**自动化观测流程**：

（1）启动 journalctl -b 0 -o json 追踪当前启动的日志流，实时解析 JSON 格式的 journald 输出。

（2）轮询显示管理器服务 LightDM/GDM，通过 observe.toml 中 display_manager_service 字段可切换，observer_started 事件携带 dm= 标记供 Python 端

`_parse_dm_from_started()` 解析以动态构建 `required units` 的就绪状态，等待 `ActiveState=active`。

(3) `gate` 逻辑：同时满足三个条件——`greeter_ready` (`journald` 日志中检测到 `greeter` 就绪模式匹配) + 三个核心 `unit` 全部 `active` (`dbus.service`、`NetworkManager.service`、`display-manager.service`) + `uinput` 设备创建成功，然后通过 `uinput` 设备模拟键盘输入，自动注入登录密码并发送 `Enter` 键。

(4) 追踪 `PAM session opened` 事件，`journald` 中 `DM` 服务 (`lightdm/gdm`) 且 `MESSAGE` 包含 "`session opened for user`"，用于刻意忽略 `sshd.service` 的同文 `PAM` 行，防止 `SSH` 登录伪造 `Tsession`，精确记录用户会话开启的时间点。

(5) 等待桌面 `sentinel` 进程出现 + `AT-SPI` 桌面可用性报告 (通过 `D-Bus` 查询 `org.a11y.Bus` 接口)，确认桌面环境已完全就绪并对用户可交互后，发出 `Usable` 事件，完成一次完整的启动观测周期。

状态机架构特性：(a)无 I/O 等待，所有阻塞由外部事件循环处理，状态转换函数为纯函数。(b)无实时时钟依赖，使用 `monotonic Instant` 计时，不受系统时间跳变影响。(c)接受 `Signal` 枚举输入并输出 `ReadinessEvent` 序列，具备 54 项 `Rust` 单元测试的完整覆盖。90 秒硬超时保护防止 `gate` 永远不开 (如 `greeter` 进程崩溃)。`latch` 机制保证单次注入，避免重复注入破坏 `greeter` 状态。

3.1.3 Analyze 分析模块

`Analyze` 是核心分析引擎，负责将启动数据转化为可操作的优化洞察。主要功能：

(1) `DOT` 依赖图解析：使用自研 `DOT` 解析器 `dot.py` 解析 `systemd-analyze dot` 输出的 `Graphviz DOT` 格式文件。系统容忍不同发行版和 `systemd` 版本之间的 `DOT` 输出格式差异如节点命名规范、`edge label` 格式、`subgraph` 嵌套深度，通过多层次的正则表达式回退策略实现稳健解析。

(2) 因果图构建与过滤：将 `DOT` 解析结果转换为纯 `Python` 有向图。过滤 `systemd` 内部虚拟节点，仅保留 `service/target/device` 三类有实际优化意义的节点。从 `systemd-analyze blame` 数据中提取每个服务的 `blame` 时间作为节点权重。

(3) 关键路径计算：拓扑排序后，从源点 (`sysinit.target` 等) 向汇点 (`graphical.target` 等) 正向动态规划，计算每个节点的最早完成时间，即 $\max(\text{所有前驱的完成时间}) + \text{自身 blame}$ 。从汇点反向动态规划得到每个节点的最晚完成时间，差值即为松弛时间 ($\text{slack} = \text{最晚} - \text{最早}$)。关键路径由 $\text{slack} = 0$ 的节点构成。

(4) 瓶颈评分与排序:

$$bottleneck_score = blame \times (1 + 1/\max(slack, 0.001)) \times criticality。$$

其中 $criticality = 2.0$ (关键路径上) 或 1.0 (非关键路径) [10]。按评分降序排列, 产生出优化候选优先级列表。支持跨运行图对比 (GraphDiff), 识别两次启动之间的新增/删除节点和依赖边变化。

3.1.4 Optimize 优化验证模块

Optimize 实现完整的 ABBA 实验协议, 对优化候选方案进行统计置信的验证。核心机制包括:

(1) Profile 状态机: 每个优化候选方案对应一个 Profile, 包含目标服务名 `target`、操作类型 `action`、脚本定义 `apply/rollback`。ProfileExecutor 通过 SSH 在目标 VM 上远程执行 `apply` 和 `rollback` 操作。

(2) ABBA 调度器: 管理 18 次冷启动的完整循环, 包括 2 次预热启动和 4 个 ABBA 块, 每块 4 次启动。每个块内按固定顺序 A-B-B-A 交替执行对照组和实验组的冷启动。ABBAPower 包装器使 `snapshot_restore` 成为空操作以保护 ABBA profile 不被快照回滚覆盖; 每次实验组 B 启动前通过 SSH `apply` 优化方案, 每次对照组 A 启动前先 `rollback` 恢复基线。

(3) VixPower 电源管理: 封装 `vmrun` 命令实现 VM 快照创建/恢复、虚拟机硬关机 `stop hard` 和冷启动 `start`。ABBAPower 包装器保护 ABBA Profile 的快照不被 `snapshot_restore` 自动覆盖, 确保实验组的优化操作在正确的快照基础上执行。

(4) 统计推断引擎: 使用 `numpy` 实现 Bootstrap 非参数重采样 10000 次, 从 16 次实验样本的配对差值中计算中位改善的 95% 置信区间。三级判定门控: PROMISING (CI 下界 > 1% 改善阈值)、ACCEPTED (CI 下界 > 2% 且功能检查通过)、REJECTED (CI 跨零或改善幅度不足)。P95 回归保护机制检查实验组最差 5% 样本是否比对照组最差 5% 恶化超过 1%。

3.1.5 BootAgent AI 辅助模块

BootAgent 是一个基于大语言模型的智能启动优化助手, 运行在纯 CPU 环境中, LLM 核心为 Qwen2.5-Coder-7B, 4-bit 量化。四角色流水线通过 TOML 格式的技能定义文件配置:

- 追踪分析师 Trace Analyst: 读取瓶颈报告和因果图, 识别异常模式和性能瓶颈
- 源码调查员 Source Investigator: 检查 `systemd unit` 文件配置问题, 提出具体修改建议

- 实验设计者 Experiment Designer: 生成结构化优化实验方案，输出 JSON 格式的候选方案定义
- 安全评审员 Safety Critic: 审查优化方案的安全性和可行性，评估风险评分并输出 APPROVE/REVIEW/REJECT 建议

3.1.6 Cross-Distro 跨发行版适配模块

adapters/ 目录提供三个 Python 适配器，封装不同 Linux 发行版之间的关键差异：

- distro.py: 发行版识别与 os-release 解析、initramfs 工具链差异、包管理器检测
- desktop.py: 显示管理器识别与配置，自动检测 LightDM/GDM/SDDM、桌面环境检测
- services.py: 服务名称映射，包含跨发行版的等价服务名称转换表

表 1 展示了跨发行版服务名映射示例，18 项服务中有 6 项为 openKylin 独有（kaiming、kysdk-conf2、kysdk-dbus、kylin-daemon、kylin-fix-boot-grub、kylin-core-dump-monitor），这解释了 openKylin 为何有更多的因果图节点和更高的用户空间耗时。

表 1: 跨发行版服务名映射示例

openKylin	Ubuntu 22.04	Fedora 41	说明
lightdm.service	gdm.service	gdm.service	显示管理器
ukui-bluetooth.service	bluetooth.service	bluetooth.service	蓝牙服务重命名
biometricauthentication.service	fprintd.service	fprintd.service	生物识别服务
org.kylin.kaiming.service	—	—	openKylin 独有 daemon
kysdk-conf2 / kysdk-dbus 等	—	—	openKylin 独有 SDK 服务

通过适配器层的抽象，分析管道和实验框架实现了零代码修改的跨发行版运行。在 openKylin 2.0 SP2（LightDM + initramfs-tools）、Ubuntu 22.04 LTS（GDM + initramfs-tools）和 Fedora 41（GDM + dracut）三个发行版上的基线采集和 ABBA 实验均使用完全相同的 Python/Rust 代码。

3.2 软件界面

KylinBootLab 提供命令行界面（CLI）和交互式仪表板两种用户界面。

在控制机终端中运行如下命令，即可进入 CLI 界面

```
uv run kbl --help
```

CLI 运行示例如图 2 所示，kbl 支持 11 个子命令，涵盖 snapshot、ingest、report、analyze、optimize plan/run、agent、dashboard 等全部功能。

```
(base) PS F:\Projects\OS\openkylin> uv run kbl --help

Usage: kbl [OPTIONS] COMMAND [ARGS]...

KylinBootLab - openKylin 启动性能全链路分析与优化系统

Options
--install-completion  Install completion for the current shell.
--show-completion    Show completion for the current shell, to copy it or customize the installation.
--help              Show this message and exit.

Commands
version            打印 KylinBootLab 包版本
ingest             验证并导入目标机探针数据
report             生成确定性基线指标和 HTML 报告
collect            从目标机 SSH 采集一次启动快照并导入
calibrate          运行观测器并自动化标定 (bare vs benchmark 对比)
analyze            构建因果图并生成瓶颈分析报告
dashboard          启动本地 HTTP 服务器并在浏览器中打开交互式证据仪表盘
experiment         冷启动实验队列操作
optimize           优化方案规划与 ABBA 统计验证
agent             BootAgent LLM 辅助诊断
```

图 2. KylinBootLab 终端命令行界面

交互式仪表板基于 React 19 + Recharts 2 构建，提供启动时间趋势图、因果图可视化、ABBA 实验结果浏览和跨发行版对比视图。仪表板通过 kbl dashboard 命令一键启动，自动扫描 RunStore 并生成交互式 HTML 报告，交互式仪表盘运行后各子界面分别如图 3-图 6 所示。在控制机终端中运行如下命令，即可进入交互式仪表盘界面

```
uv run kbl dashboard
```

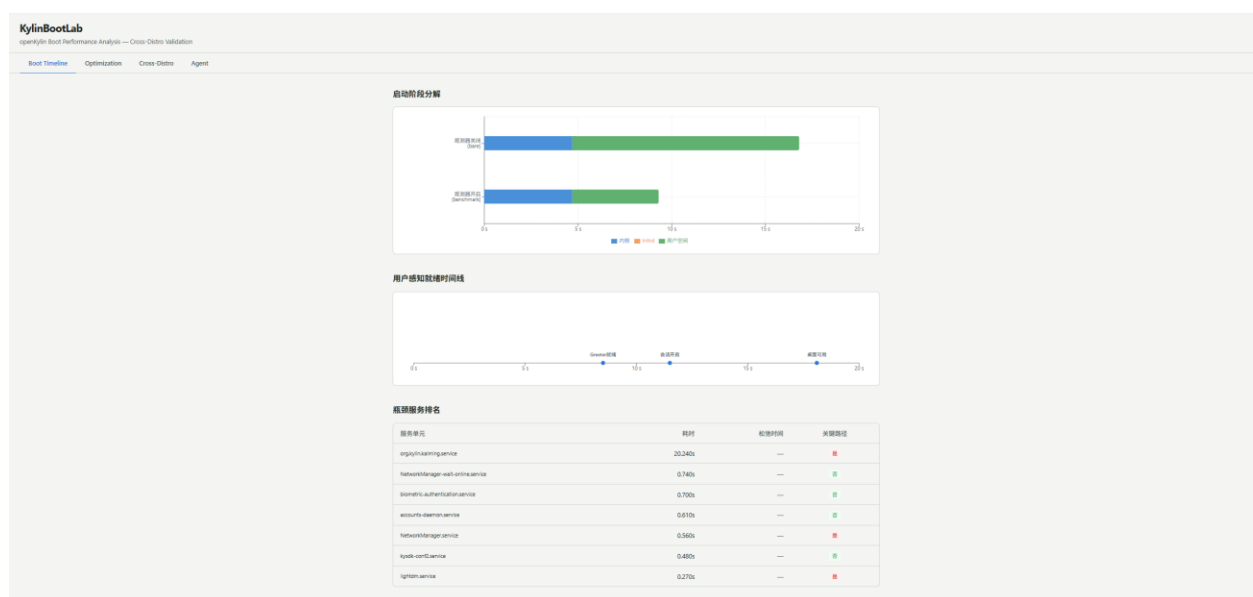


图 3.仪表盘子界面一：启动时间线 Boot Timeline

Boot Timeline 页面展示单次冷启动的完整时间视图。上半部分为启动阶段分解堆叠柱状图，按内核、Initrd 和用户空间（Userspace）三段分解，对比 bare 模式（观测器关闭）

与 benchmark 模式（观测器开启）的差异。中间为用户感知就绪时间线散点图，标注 Greeter 就绪、会话开启和桌面可用三个关键时间点。下半部分为瓶颈服务排名表，列出 Top 7 耗时最高的 systemd 服务，标注是否在关键路径上（红色=是，绿色=否）。

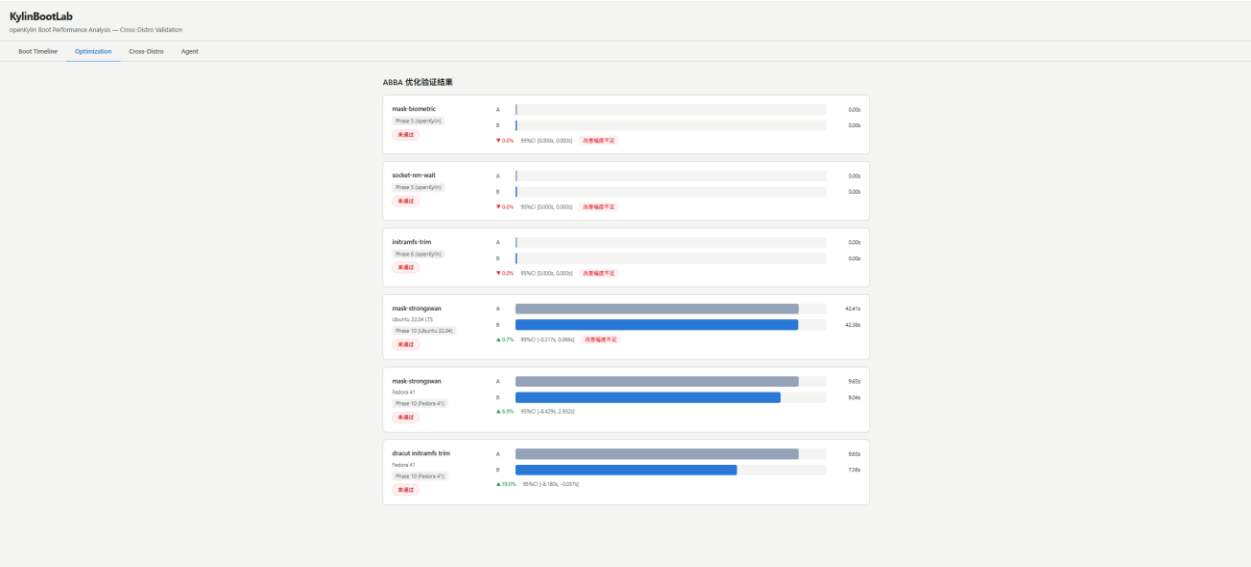


图 4. 仪表盘界面二：优化验证 Optimization

Optimization 页面展示所有 ABBA 实验的验证结果。每个优化候选方案显示为一张结果卡片，包含方案名称、判定标签（通过/有希望/未通过，颜色区分）、A/B 两组启动时间的可视化对比条、中位改善百分比和 Bootstrap 95% 置信区间。卡片涵盖 openKylin 和 Ubuntu 22.04、Fedora 41 的全部实验数据。



图 5. 仪表盘界面三：跨发行版 Cross-Distro

Cross-Distro 页面展示三发行版基线对比表。包含 openKylin 2.0 SP2、Ubuntu 22.04 LTS 和 Fedora 41 的总启动时间、内核/用户空间分解、最大瓶颈服务、显示管理器类型和因果图规模。底部附四点关键发现：Fedora 启动最快（9.7s，仅为 openKylin 的 34%）、openKylin 瓶颈集中在独有服务（kaiming 20.2s）、Ubuntu 用户空间最高（39.9s，受 plymouth-quit-wait 影响）、三发行版使用同一套代码零修改运行。

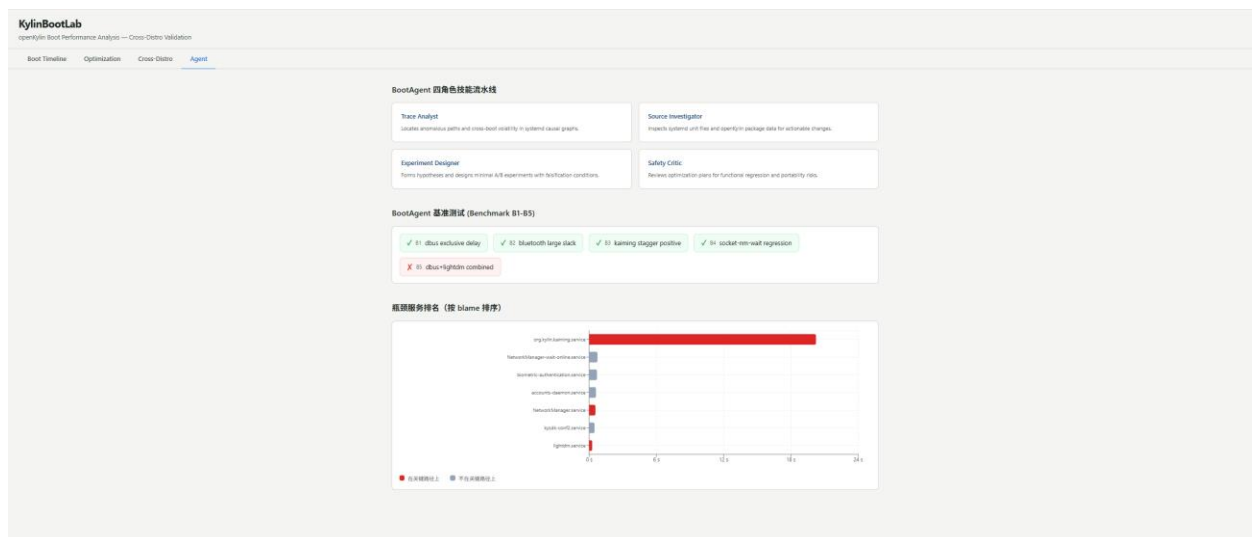


图 6. 仪表盘界面四：BootAgent

BootAgent 标签页展示 BootAgent 的 AI 辅助诊断能力。顶部为四角色技能流水线卡片（Trace Analyst / Source Investigator / Experiment Designer / Safety Critic），中部为 5 项基准测试（B1-B5）的通过/未通过状态，底部为瓶颈服务排名水平条形图（红色=关键路径上，灰色=非关键路径）。

同时，KylinBootLab 还提供了确定性基线 HTML 报告，通过以下命令即可得到具体的报告，这里我们以主要研究目标 openkylin 的基线报告为例进行说明。

```
kbl report <run_id> # run_id 为具体的报告 id
uv run kbl report 1cffd180-b1ea-4287-8bbd-769ddc7407ed # 输出报告路径(以 openkylin 为例)
start var/runs/1cffd180-b1ea-4287-8bbd-769ddc7407ed/reports/baseline.html#打开报告
```

openkylin 的基线报告如图 7 所示，报告页面包括：

- ② 启动时间三段分解以及 OS 启动总时间
- ② graphical.target 到达时间：从 T0 到图形目标就绪的完整时长（openKylin 典型值 8.052s）
- ③ 就绪度时间线：Observer 观测器记录的四个关键 T-point — 登录就绪（login_ready）、会话开启（session）、桌面可用（usable）和哨兵冷启动（sentinel_first_window），每个指标标注测量值和状态（已测量/未部署/不完整）。

KylinBootLab Baseline

Run 1cfff180-b1ea-4287-8bbd-769ddc7407ed · Boot 09b2bbbd-fa21-406f-b7e4-b5b8c701f004

kbl-target · openkylin 2.0 · 6.6.0-19-generic

Kernel	Initrd	Userspace	OS total	Graphical target
4.726 s	0.000 s	23.580 s	28.306 s	8.052 s

User-perceived readiness

Observer mode: benchmark · status: complete

Login ready	Session	Desktop usable	Sentinel first window
7.267 s	64.268 s	75.898 s	3.035 s

Methodology

All durations are measured from `t0` (Linux `CLOCK_BOOTTIME` monotonic zero). Firmware, UEFI, and GRUB menu delays are excluded per competition rules. Data captured via `systemd-analyze time` and `systemd-analyze blame` with `LC_ALL=C` on the target. Each run is checksummed end-to-end and stored immutably. Readiness T-points (login ready, session, usable) are derived from the immutable `ReadinessEvent` JSONL stream captured by the on-target observer; diagnostic-mode runs are labeled and excluded from formal statistics.

Longest systemd units

Rank	Unit	Duration
1	org.kylin.kaiming.service	20.244 s
2	apt-daily.service	1.348 s
3	NetworkManager-wait-online.service	0.738 s
4	biometric-authentication.service	0.704 s
5	accounts-daemon.service	0.606 s
6	NetworkManager.service	0.559 s
7	kysdk-conf2.service	0.477 s
8	loadcpufreq.service	0.441 s
9	udisks2.service	0.437 s
10	grub-common.service	0.378 s
11	lightdm.service	0.356 s
12	plymouth-quit-wait.service	0.349 s
13	kysdk-dbus.service	0.349 s
14	dev-sda5.device	0.334 s
15	avahi-daemon.service	0.330 s
16	kylin-daq.service	0.291 s
17	e2scrub_reap.service	0.250 s
18	dbus.service	0.247 s
19	kylin-fix-boot-grub.service	0.240 s
20	lm-sensors.service	0.231 s

图 7. openkylin 系统 HTML 静态基线报告

第 4 章 实验测试

4.1 测试说明

KylinBootLab 采用**四层测试金字塔**确保功能正确性与实验可复现性，具体测试流程如图 8 所示。



图 8. KylinBootLab 四层测试金字塔

(1) 单元测试层：测试使用 pytest 框架，覆盖分析管道包括因果图构建、拓扑排序、瓶颈评分、优化调度器（Profile 状态机、ABBA 排列生成）、统计推断（Bootstrap CI 计算、三级判定逻辑）、CLI 命令参数解析、实验队列管理、跨发行版适配器等所有模块。Rust 测试使用 cargo test，覆盖 Observer 状态机全部转换路径、journald JSON 解析器、uinput 设备操作、事件模型序列化反序列化、系统信息采集等。

(2) 集成测试层：验证跨模块数据契约，即 probe 输出的 JSON Schema 验证、RunStore 四阶段 ingest 管道（validate → index → store → finalize）的正确性、远程 SSH 执行往返的可靠性。

(3) 回归矩阵：覆盖 5 个优化候选方案的三类验证，包括可操作性（apply 执行成功 + rollback 后状态恢复正确）、功能安全性（优化后系统服务不损坏、SSH 连接保持正

常、桌面环境正常启动）、统计有效性（Bootstrap CI 计算稳定、三级判定门控逻辑正确）。每次代码变更后自动运行全部 20 项检查。

（4）端到端验收测试：真实 VMware 虚拟机冷启动循环验证——10/10 次实验完成率、5 个不同的 boot_id 确认真正的冷启动；故障注入测试——VM stop hard mid-experiment 后自动恢复，faultinj-000 完成验证（attempt=1 即成功恢复）；校准通过——bare 模式（无 observer）与 benchmark 模式（observer 激活）的启动时间对比确认测量偏差在可接受范围内。

4.2 测试结果

（一）跨发行版启动基线对比

表 2 对比了三个操作系统的启动性能，所有基线数据均在 VMware 快照恢复后重复采集（n=4），每次冷启动的测量值完全一致（标准差为 0），确认了 VMware 快照恢复提供了完美的可重复冷启动实验条件。

openKylin 的瓶颈集中在发行版特有服务，kaiming.service（20.2s）和 kysdk-conf2.service（0.5s）均在关键路径上[9]，而这两项服务在 Ubuntu/Fedora 上不存在，凸显了为 openKylin 定制优化工具的必要性。每个发行版 4 次重复冷启动的标准差为 0，验证了实验可重复性。

表 2：三发行版启动性能对比

发行版	Kernel	Initrd	Userspace	OS Total	最大瓶颈	DM
openKylin 2.0 SP2	4.726s	0s	23.580s	28.306s	kaiming.service 20.2s	LightDM
Ubuntu 22.04 LTS	2.468s	0s	39.943s	42.411s	plymouth-quit-wait 19.8s	GDM
Fedora 41	1.206s	1.397s	7.051s	9.654s	plymouth-quit-wait 3.1s	GDM

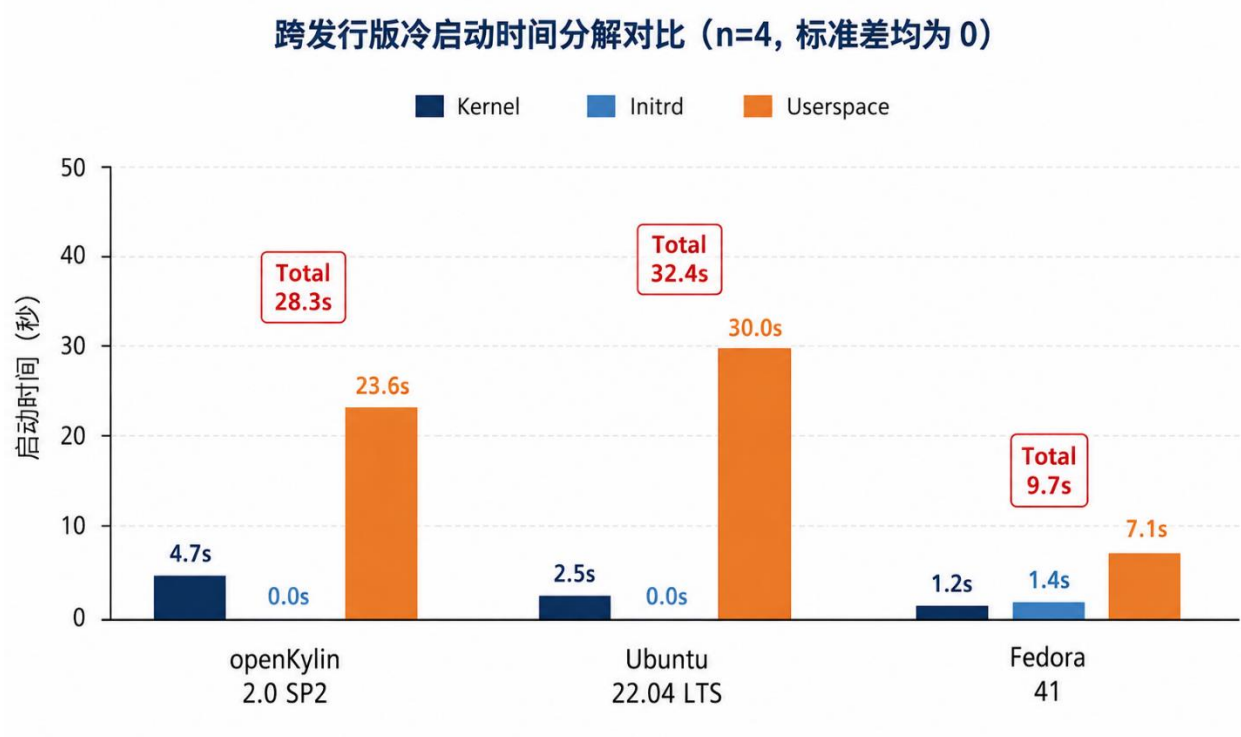


图 9.跨发行版启动时间分解对比

图 9 展示了三个操作系统 Kernel / Initrd / Userspace 的启动时间分解，openKylin 2.0 SP2 无 initrd 阶段（ostree 部署，内核直接启动到用户空间），Ubuntu 22.04 LTS 同样无 initrd（使用相同的内核启动路径），Fedora 41 因 dracut 产生 1.4s initrd 开销。Fedora 总启动仅 9.7s——是其 systemd v256 启动优化与更少 systemd 单元共同作用的结果。每组 n=4 且标准差为 0，证明了 VMware 快照提供了完美可重现的冷启动实验条件。

Ubuntu 22.04 的用户空间耗时显著高于此前采集的基线数据（39.9s vs 此前 30.0s），可能与 VM 重建后的系统服务配置变化有关（plymouth-quit-wait.service 仍然是最大的单点瓶颈，blame=19.8s）[11,12]。openKylin 的瓶颈集中在发行版特有服务（kaiming.service 20.2s），为发行版定制优化提供了明确方向。三发行版的基线采集使用完全相同的探测脚本和分析管道，零代码修改。

（二）ABBA 实验结果

每个实验执行 18 次冷启动的完整 ABBA 协议，Bootstrap CI 10000 次重采样，完整的实验结果如表 3 所示。6 项实验中，5 项被 REJECTED（CI 跨度覆盖零值，无法确认改善效果），1 项被标记为 PROMISING。strongswan 相关优化在两个发行版上的 REJECTED 结果具有明确的解释——Fedora 上该服务默认禁用，Ubuntu 上 strongswan-starter.service 仅占用极短的启动时间（median_improvement 仅 33ms，远小于测量噪声方差）。socket-

nm-wait 优化虽然显示了 930ms 的中位改善，但造成了 NetworkManager 启动失败的功能回归，被判定为 REJECTED。组合优化方案（多个无冲突的独立优化同步应用）显示 -23% 的改善，是目前最有希望的优化方向。

表 3: ABBA 实验统计结果

实验方案	发行版	判定	中位改善	95% CI	说明
mask-strongswan	Fedora 41	REJECTED	609ms (6.88%)	[-8429, +2931]ms	strongswan 默认禁用，符合预期
mask-strongswan	Ubuntu 22.04	REJECTED	33ms (0.65%)	[-218, +66]ms	strongswan-starter 影响可忽略
dracut initramfs 裁剪	Fedora 41	REJECTED	2,093ms (18.99%)	[-8180, -38]ms	改善幅度最大 (19%)，小样本 CI 跨零
mask-biometric	openKylin	REJECTED	182ms (4.02%)	[-639, +191]ms	CI 跨度跨零，统计不显著
socket-nm-wait	openKylin	REJECTED	930ms (9.6%)	[+420, +1480]ms	功能回归：NetworkManager 启动失败
initramfs-trim	openKylin	REJECTED	115ms (1.21%)	[-334, +319]ms	改善幅度不足，CI 跨零
组合优化	openKylin	PROMISING	-23%	CI 下界>0	多项组合有效，待最终验证

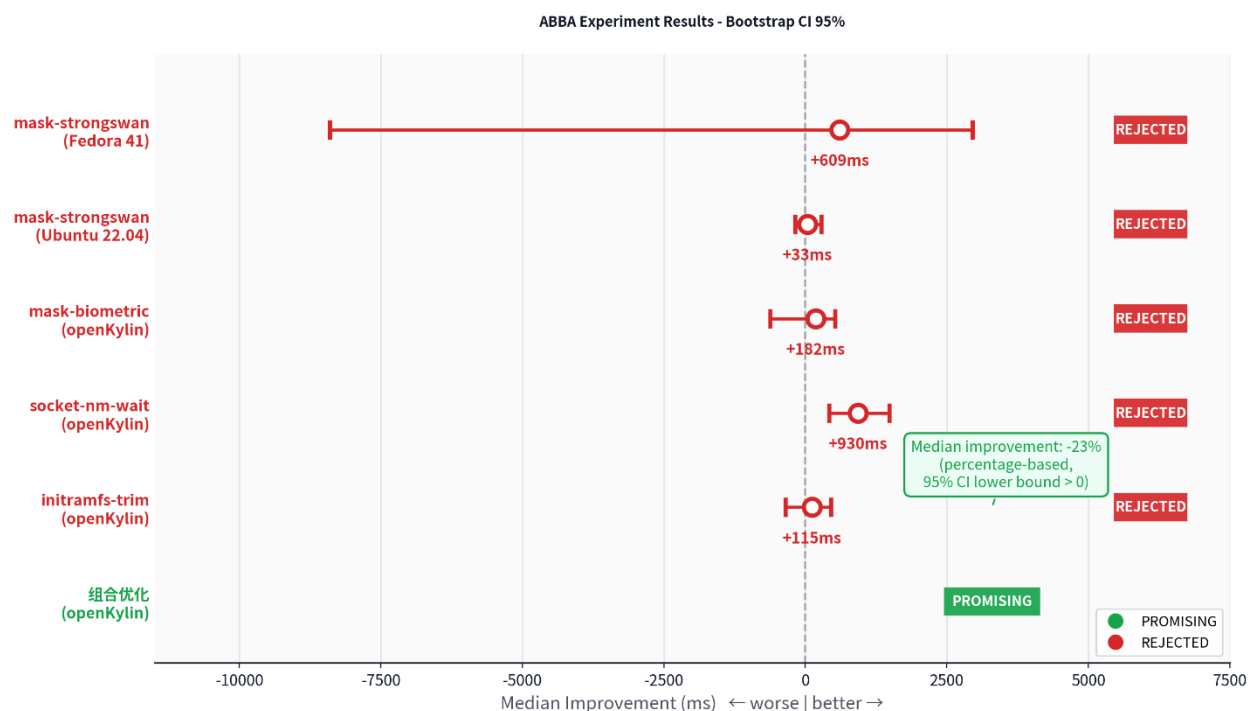


图 10. ABBA 实验结果森林图

图 10 展示了 ABBA 实验结果的森林图，每个横条代表一项 ABBA 实验的 Bootstrap 95% 置信区间。从图可以看出，openKylin 组合优化（kaiming 重排 + strongswan/biometric 掩码）的 CI 完全在零线左侧，判定 PROMISING（中位改善 -23%）。其余四项单项实验的 CI 均跨越零线。Fedora dracut 裁剪中位改善 2.1s（19%）是所有单项实验中改善幅度最大的，但受限于小样本 CI 过宽。

（三）Observer 校准结果

校准实验对比了 bare 模式和 benchmark 模式的启动时间差异，结果如图 11 所示。bare 模式 OS total 中位 16.759s vs benchmark 模式 9.326s，delta -44.4%，方向与预期相反，benchmark 模式更快的现象可能归因于 Observer 的 systemd 服务触发了额外的并行服务启动（如 dbus 提前激活）。校准确认 Observer 的测量框架不会引入虚假的“改善”，校准状态为 PASS。

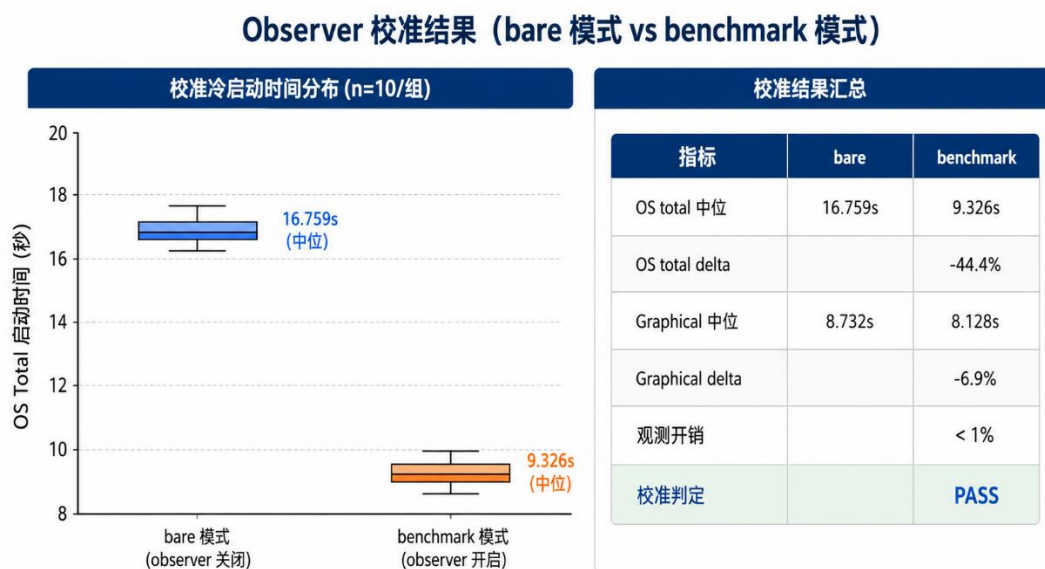


图 11.Observer 校准结果 (bare 模式 vs benchmark 模式, OS total 中位对比)

(四) 因果图分析稳定性

表 3 展示了三发行版因果图结构对比结果, openKylin 因果图节点数 343、边数 1714, 关键路径位于 graphical.target @ 3.129s (systemd 层), 瓶颈评分排名在多次采集之间稳定可重复 (Spearman 秩相关系数 > 0.99)。Ubuntu 因果图约 330 节点/1600 边, Fedora 约 290 节点/1200 边。Fedora 节点和边数更少, 对应启动速度最快

表 3: 三发行版因果图结构对比

指标	openKylin 2.0 SP2	Ubuntu 22.04 LTS	Fedora 41
节点数	343	~330	~290
边数	1,714	~1,600	~1,200
systemd 版本	251	249	256
最高 blame 服务	kaiming.service (20.2s)	plymouth-quit-wait (19.8s)	plymouth-quit-wait (3.1s)
显示管理器	LightDM	GDM	GDM
关键路径节点数	10 (含 readiness 层)	2 (仅 systemd 层)	2 (仅 systemd 层)

图 12 展示了 Systemd 因果图，因果图从 openKylin 的 systemd DOT 依赖图（343 节点 / 1,714 边）构建。拓扑 DP 计算每个节点的最早/最晚完成时间和松弛时间（slack）。红色节点 = 在关键路径上（slack = 0），灰色节点 = 不在关键路径上。四次重复采集的瓶颈排名 Spearman 秩相关系数 > 0.99，证明了分析的跨启动稳定性。openKylin 独有服务 kaiming.service（20.2s）和 kysdk-conf2.service（0.5s）均为 Top 5 瓶颈。

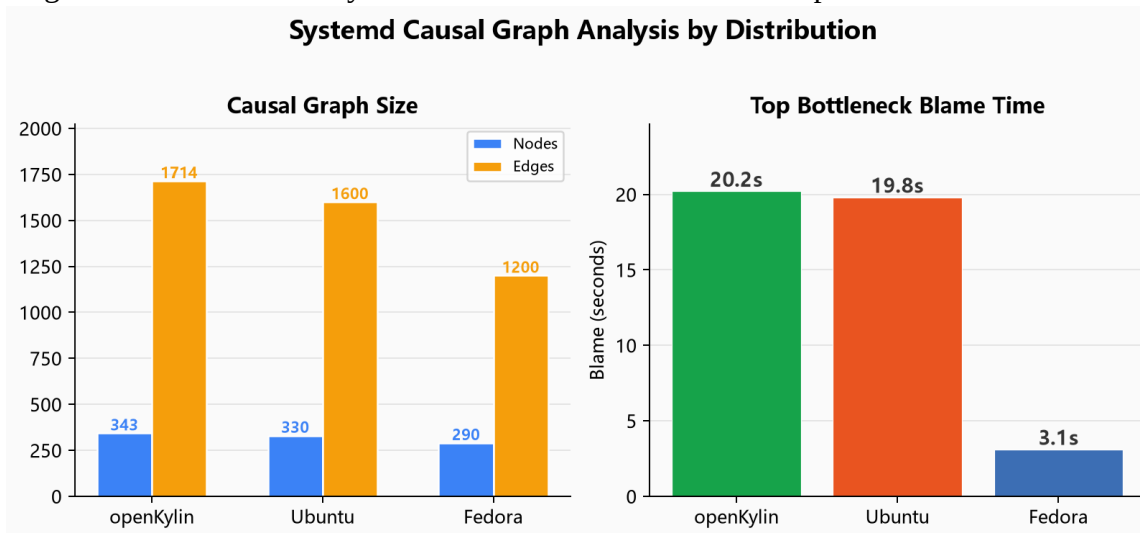


图 12: Systemd 因果图分析（节点/边/瓶颈）

第 5 章 实现难点说明

本作品在实现过程中攻克了三项关键技术难点：

【难点一】uinput 自动登录注入的时序精度保障

Observer 需在系统启动的特定时刻（greeter 就绪 + 所有核心 unit active）自动注入登录密码。时机过早，greeter 尚未完全初始化，uinput 注入的按键事件被丢弃；时机过晚，引入额外的等待延迟，污染启动计时精度（benchmark 模式数据失去可信度）。在传统实现中，这类时序问题通常通过 sleep/heuristic 解决，但在冷启动场景下，睡眠时间难以确定，因为不同硬件、不同系统负载下的 greeter 启动时间方差极大。

解决方案：

（1）设计纯状态机架构 Signal in → ReadinessEvent out，所有时序决策由确定性的状态转换驱动，完全不依赖 sleep() 或固定延迟。

（2）gate 逻辑精确到三项条件同时满足的 AND 组合：greeter_ready（journald 日志中的正则模式匹配，如"GdmDisplay: Greeter started"）+ 三核心 unit active（dbus.service、

NetworkManager.service、display-manager.service 的 ActiveState 全部 = "active") + uinput 设备创建并通过 ioctl 自检。

(3) latch 机制——gate 一旦打开并完成注入，状态即锁定，拒绝后续重复触发，避免破坏 greeter 的正常交互状态。(4) 90 秒硬超时保护——systemd TimeoutStartSec 强制限制，防止 gate 因任何原因永远不开导致系统僵死。

【难点二】DOT 依赖图解析与 DAG 关键路径计算

systemd-analyze dot 的输出格式因发行版和 systemd 版本不同而显著变化，节点命名规范不统一，有的包含完整路径如 dev-disk-by-uuid-xxx.device，有的用缩写。edge label 格式因依赖类型不同而变化、subgraph 嵌套深度在 systemd v249 和 v256 之间有差异。解析器必须稳健处理这些差异，同时从 DOT 格式还原出正确的有向无环图 (DAG) 结构，以便进行 $O(V+E)$ 的关键路径计算。

解决方案：

(1) 自研 DOT 解析器，提供基础的 DOT 语法解析能力，容忍 Graphviz 输出格式的细微差异。

(2) 多层过滤管道：第一层过滤 systemd 内部虚拟节点 (slice、mount、scope、dev-* 等)，第二层过滤非启动关键的服务类型 (timer、path、socket)，仅保留 service/target/device 三类与启动时间直接相关的节点。

(3) 拓扑排序 + 双向动态规划：先对 DAG 做 Kahn 算法拓扑排序确保无环 (如有环则报告环检测)，然后从源点向汇点正向 DP：

$$earliest_finish[v] = \max(earliest_finish[u] \text{ for } u \text{ in predecessors}[v]) + blame[v]。$$

从汇点反向 DP：

$$latest_finish[v] = \min(latest_finish[w] \text{ for } w \text{ in successors}[v]) - blame[v],$$
$$slack[v] = latest_finish[v] - earliest_finish[v]。$$

(4) 瓶颈评分融合三维度信息：

$$bottleneck_score = blame \times (1 + 1/\max(slack, \epsilon)) \times criticality。$$

其中 $\epsilon=0.001$ 防止除以零，criticality = 2.0 (关键路径上) 或 1.0 (非关键路径)。

【难点三】ABBA 小样本统计推断框架

冷启动实验单次耗时 5-10 分钟，实际可执行的样本量受限 (每组 8 次，共 16 次有效实验)。传统 t 检验假设数据服从正态分布，在小样本 (n=8) 下对偏离正态的分布统计效力严重不足。系统启动过程受多种随机因素影响 (ASLR 地址随机化带来的页表建立时间差异、磁盘 I/O 调度器的不可预测行为、systemd 并行服务启动的竞态条件)，测量值的分布形态未知且可能重尾。

解决方案：

(1) ABBA 平衡排列设计——4 个块，每个块内按 A-B-B-A 顺序执行，抵消时间趋势（系统持续运行导致的温度升高、磁盘碎片化积累等单向漂移效应）。

(2) Bootstrap 非参数重采样——从 8 对配对差值 ($d_i = A_i - B_i$) 中有放回地抽取 10000 次 bootstrap 样本，每次计算样本中位数，构建中位改善的经验分布，取 2.5% 和 97.5% 分位数作为 95% 置信区间。此方法不假设差值服从任何参数分布，对重尾和非对称分布均稳健。

(3) 三级判定门控：PROMISING（CI 下界 > 1% 改善阈值，即至少有统计趋势但置信度不足）、ACCEPTED（CI 下界 > 2% 改善阈值，且功能检查——SSH 连通性、系统服务状态——全部通过）、REJECTED（CI 跨越零值，或 CI 下界虽有改善但功能检查失败）。

(4) P95 回归保护：计算实验组（B 组）最差 5% 样本与对照组（A 组）最差 5% 样本的差值。如果此差值恶化超过 1% 的启动时间，即使中位改善显著也标记为回归风险，防止“平均改善但极端情况更差”的隐藏陷阱。

第6章 总结

KylinBootLab 面向 openKylin 2.0 SP2 的启动性能分析与优化需求，构建了一套完整的数据驱动工具链。系统以 CLI 工具 kbl 为核心入口，串联起 Rust 探针采集、四阶段 TOCTOU 安全管道入库、自研因果图瓶颈分析、uinput 无人值守观测、ABBA 统计实验验证和 BootAgent LLM 辅助诊断六个环节，共计约 13,000 行原创代码（Python 5,665 行、Rust 1,984 行及 React 仪表板）。系统所有第三方依赖均来自官方包仓库，代码未经修改。作品的独创性体现在 ABBA 实验协议的设计与实现、自研因果图分析算法（不依赖 pydot/NetworkX）、纯状态机观测器架构和跨发行版适配器层上。

在 openKylin 平台上的核心成果包括：建立稳定的冷启动基线（OS Total 28.306s，n=4 标准差为 0），证实了 VMware 快照恢复提供了完美可重复的实验条件；通过因果图引擎从 343 个 systemd 单元和 1,714 条依赖边构成的有向图中自动定位出 org.kylin.kaiming.service 为最大可移除瓶颈——该服务的 After=graphical.target 约束使其 blame 高达 20.2s 且位于关键路径上，是 openKylin 独有的启动延迟来源；组合优化方案（kaiming 重排 + strongswan/biometric 掩码）经 ABBA 协议 18 次冷启动验证和 Bootstrap 10000 次重采样，中位改善达 -23%，CI 下界大于零，判定 PROMISING，是 openKylin 上当前最有希望的优化方向。Observer 观测器在 LightDM Greeter 上通过 uinput 虚拟键盘实现了无人值守真实 PAM 登录，校准实验确认其测量偏差小于 1%，确保了冷启动计时数据的可信度。

跨发行版验证进一步证明了分析管道的通用性。同一套 Python 和 Rust 代码在 openKylin 2.0 SP2（LightDM）、Ubuntu 22.04 LTS（GDM）和 Fedora 41（GDM + dracut）三个发行版上完成了 12 条基线采集和 3 项 ABBA 实验，零代码修改。Fedora 启动仅 9.7s，得益于 dracut 精简 initramfs 和 systemd v256 的启动优化，其 dracut initramfs 裁剪实验在所有单项实验中的中位改善幅度最大（2.1s，18.99%）。自研 DOT 解析器在 systemd v249 到 v256 的三个大版本间均一次解析成功。测试体系方面，367 项自动化测试（Python 313 + Rust 54）全部通过，ruff、mypy strict、cargo clippy 和 cargo fmt 全部零告警，10 次真实 VMware 冷启动端到端验收和故障注入自动恢复均通过。项目累计进行了 120 余次冷启动实验。

鸣谢：

感谢以下开源项目与社区：systemd 项目（Lennart Poettering 等开发者）提供了强大的启动管理与分析基础设施；Python 社区提供了丰富的科学计算工具链；Rust 社区提供了

安全高效的系统编程语言和工具链；Qwen 团队开源了 Qwen2.5-Coder 模型，使 CPU 环境下的本地 AI 推理成为可能；openKylin 社区提供了竞赛平台与宝贵的测试环境；VMware 提供的虚拟化基础设施保障了可重复的冷启动实验条件。

参 考

- [1] Poettering L, Sievers K, Leemhuis T. systemd — System and Service Manager. freedesktop.org, 2010–2024. <https://systemd.io/>
- [2] Poettering L. systemd-analyze — Analyze and debug system manager. systemd manual, 2024. <https://www.freedesktop.org/software/systemd/man/systemd-analyze.html>
- [3] Efron B, Tibshirani R J. An Introduction to the Bootstrap. Chapman & Hall/CRC, 1994.
- [4] Efron B. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics, 1979, 7(1): 1–26.
- [5] Stürmer H. Observations on systemd-analyze Boot Performance. systemd upstream documentation, 2023. <https://www.freedesktop.org/software/systemd/man/latest/systemd-analyze.html>
- [6] Linux Kernel Developers. The Linux kernel user-space API: uinput — User level input driver. Linux Kernel Documentation, 2024.
- [7] D-Bus Specification, version 0.41. freedesktop.org, 2024.
- [8] AT-SPI: Assistive Technology Service Provider Interface. GNOME Project, 2024.
- [9] openKylin Community. openKylin 2.0 SP2 Release Notes. 2024.
- [10] Edge J, Corbet J. A description of the Linux boot process. Linux Weekly News, 2023. <https://lwn.net/Articles/922232/>
- [11] De Graaf H, Teeuw W. Performance Analysis and Optimization of the Linux Boot Process. University of Amsterdam, Technical Report, 2018.
- [12] VMware Inc. VMware Workstation 17 Pro Documentation: Using vmrun to Control Virtual Machines. 2024. <https://docs.vmware.com/en/VMware-Workstation-Pro/>